

INTERNAL · TIM ENGINEERING

Panduan Onboarding AutoGrade

Gambaran sistem, alur data, struktur repositori, dan aturan kerja bagi anggota tim baru, manusia maupun AI agent.

Sistem

Kamera · AI · Konsol operator

Integrasi

AutoERP · PLC · Timbangan

Pembaca

Developer · AI / Vision · IoT

Informasi Dokumen

Judul	Panduan Onboarding AutoGrade
Versi	1.1
Tanggal	15 September 2026
Klasifikasi	Internal, tidak untuk dibagikan ke pihak luar
Pemilik	Tim Engineering AutoGrade
Naskah sumber	docs/ONBOARDING.md

Daftar Isi

1. Gambaran Umum	3
2. Arsitektur: Dua Sistem Terpisah	4
3. Komponen yang Berjalan	5
4. Alur Data: Satu Janjang	6
5. Alur Kunjungan Truk	8
6. Struktur Repositori	9
Folder utama	9
Isi folder src/palmgrade	9
Berkas di akar repositori	11
7. Menjalankan Sistem	12
Di Mac: pengembangan konsol	12
Di PC pabrik: Linux dengan GPU	12
8. Hal yang Wajib Diwaspadai	13
9. Aturan Kerja Tim	14
10. Rujukan Dokumen	15
11. Glosarium	16
12. Langkah Pertama	17
Riwayat Revisi	18

Dokumen ini ditujukan bagi anggota tim yang baru pertama kali bekerja dengan repositori `autograde`. Setelah membacanya (sekitar 20 menit) pembaca diharapkan memahami apa yang dikerjakan sistem ini, di mana setiap bagian kodenya berada, dan aturan mana yang tidak boleh dilanggar.

Catatan untuk AI agent (Codex, Claude). Dokumen ini adalah titik masuk. `CLAUDE.md` berisi ringkasan aturan yang lebih padat, dan `docs/overview.md` berisi rincian teknis. Dokumen ini memberi konteks yang menghubungkan keduanya.

1. Gambaran Umum

AutoGrade adalah sistem penilaian mutu tandan buah segar (TBS) kelapa sawit berbasis kamera dan kecerdasan buatan, yang dipasang di pabrik kelapa sawit (PKS).

Truk pemasok membawa buah sawit (disebut **janjang** atau tandan) ke pabrik. Buah dituang ke conveyor dan melintas di bawah kamera. Kamera dan model AI menilai setiap janjang: layak diterima (**ACC**) atau ditolak (**REJ**). Hasil penilaian ini menjadi dasar pembayaran kepada pemasok.

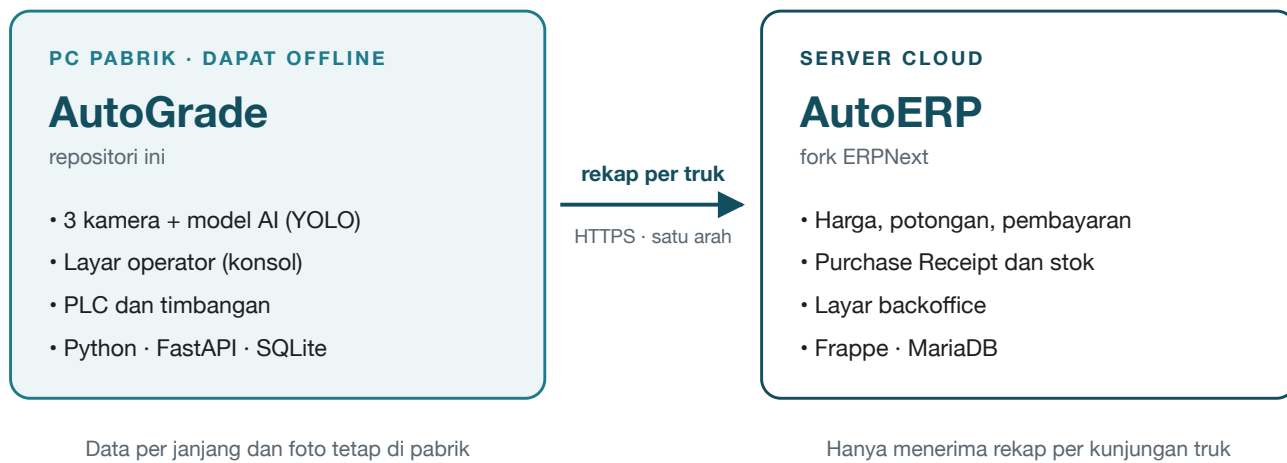
Sebelumnya penilaian dilakukan secara manual oleh petugas sortasi, sehingga hasilnya bergantung pada orang, jam kerja, dan tingkat kelelahan. AutoGrade membuat penilaian konsisten dan terdokumentasi, lengkap dengan foto setiap janjang sebagai bukti bila ada keberatan dari pemasok.

Tiga fungsi utama AutoGrade:

1. Menilai setiap janjang di tiga conveyor (disebut **line**), masing-masing dengan satu kamera.
2. Menyediakan layar operator: truk yang sedang dibongkar di tiap line, jumlah janjang, dan berat timbangan.
3. Mengirim **rekap per kunjungan truk** ke AutoERP, sistem yang mengelola pembukuan dan pembayaran.

Di luar cakupan AutoGrade: harga, potongan, pembayaran, dan faktur. Seluruhnya dikelola AutoERP. AutoGrade berperan sebagai sensor dan layar operasional.

2. Arsitektur: Dua Sistem Terpisah



Gambar 1. Pembagian peran AutoGrade di pabrik dan AutoERP di cloud

Pemisahan ini disengaja, dengan dua alasan:

- **Koneksi internet di pabrik tidak dapat diandalkan.** Seluruh kebutuhan operator harus tetap berfungsi ketika internet terputus, sehingga semuanya berjalan di PC pabrik.
- **ERP adalah buku besar, bukan penyimpan kejadian.** Mengirim ribuan data janjang per hari ke ERP membebani tanpa manfaat. ERP cukup menerima ringkasan per truk: berat dan persentase buah yang ditolak. Foto dan data per janjang tetap di pabrik sebagai bukti.

Arah komunikasi hanya satu: pabrik → ERP. ERP tidak pernah menghubungi pabrik, karena PC pabrik tidak menerima koneksi masuk dari internet.

3. Komponen yang Berjalan

Satu image Docker dijalankan empat kali dengan peran berbeda:

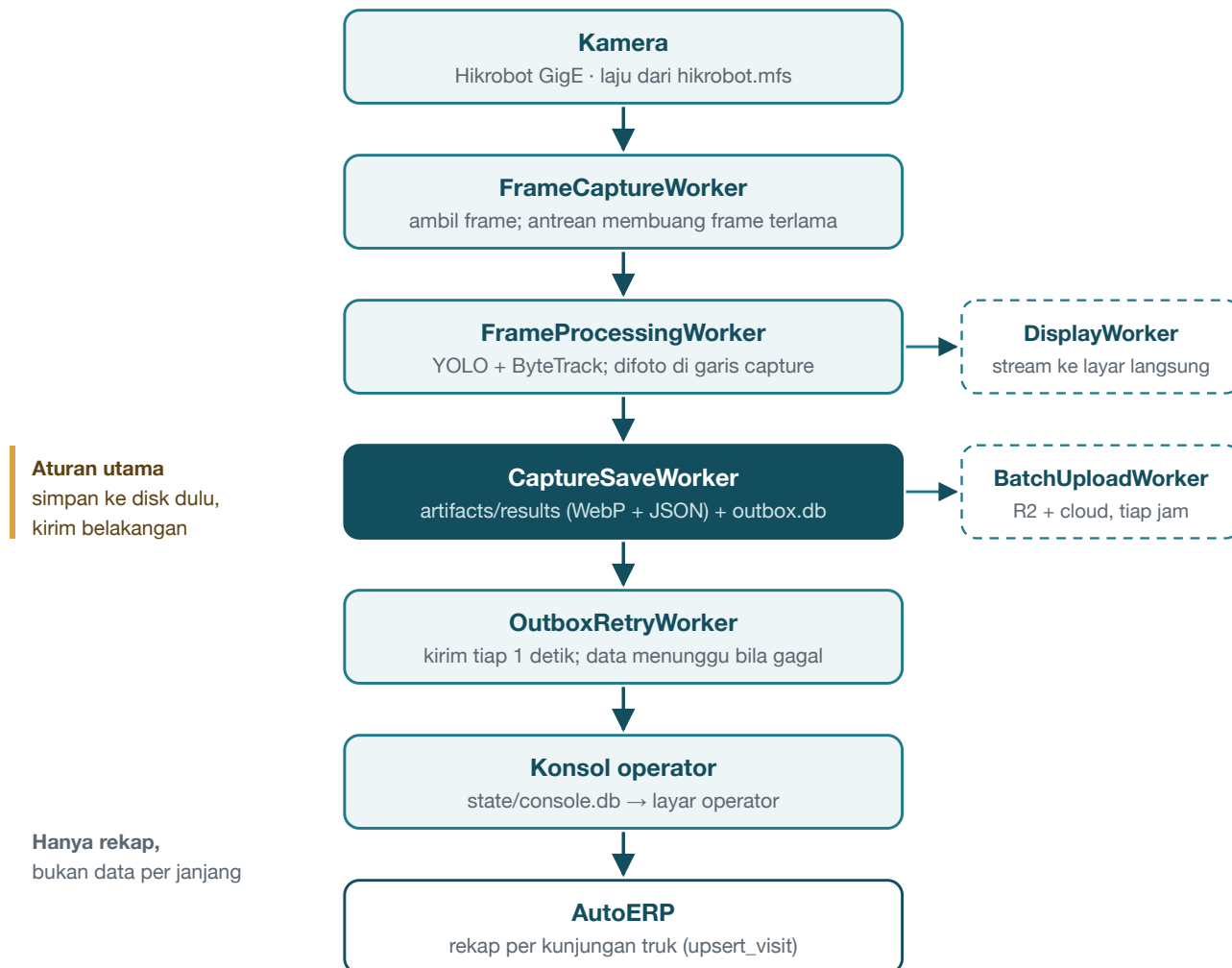
Container	Port	Fungsi	Dampak bila berhenti
<code>ripe_line_1</code>	8001	kamera dan AI line 1	line 1 berhenti menilai; line lain tetap berjalan
<code>ripe_line_2</code>	8002	kamera dan AI line 2	sama seperti di atas
<code>ripe_line_3</code>	8003	kamera dan AI line 3	sama seperti di atas
<code>palmgrade_console</code>	8000	layar operator (<code>/console</code>)	layar operator mati; line tetap menilai dan menyimpan

Line dan konsol memakai **modul aplikasi yang berbeda**: `main.py` untuk line (memuat torch, OpenCV, dan driver kamera) dan `console_main.py` untuk konsol, yang **tidak boleh** mengimpor torch maupun OpenCV. Tujuannya satu: gangguan pada kamera tidak boleh mematikan layar operator.

Layar operator terkunci. Sampai ada yang masuk dengan **email dan sandi**, seluruh layar tertutup gerbang dan seluruh API konsol menjawab 401. Akun datang dari dua tempat: dibuat di AutoERP (DocType `AutoGrade Operator` , ikut turun bareng master data) atau dibuat lokal di PC itu dengan `make operator` : akun bawaan dan akun support, supaya pabrik yang belum pernah dapat internet tetap bisa dibuka. Keduanya diperiksa di pabrik, jadi login tetap jalan saat internet mati: yang ikut turun itu hash sandinya, bukan sandinya. Sejak 2026-09-26 akun lokal juga bisa dibuat dari tab **Akun** (khusus support); akun itu cuma ada di PC tersebut, tidak naik ke AutoERP.

4. Alur Data: Satu Janjang

Bagian ini adalah inti sistem.



Gambar 2. Perjalanan data satu janjang, dari kamera sampai AutoERP

1. **Pengambilan frame.** `FrameCaptureWorker` mengambil frame dari kamera dengan laju tetap. Laju itu ditentukan oleh kamera sendiri (lihat bagian 8). Frame masuk ke antrean yang **membuang frame terlama** saat penuh, sehingga model selalu memproses gambar terbaru.
2. **Penilaian.** `FrameProcessingWorker` menjalankan YOLO dan ByteTrack. ByteTrack memastikan satu janjang yang terlihat di banyak frame berturut-turut **dihitung satu kali**. Janjang dihitung ketika kotaknya **menyentuh garis capture**, satu garis lurus yang diatur dari layar setelan. Area ROI menentukan *di mana* (bagian gambar yang dianggap conveyor); garisnya menentukan *kapan*.
3. **Simpan ke disk sebelum mengirim.** Ini aturan terpenting di repositori ini. Worker penilaian **tidak pernah** berkomunikasi lewat jaringan, dan sejak September 2026 ia juga **tidak menulis ke disk sendiri**: ia menyerahkan pekerjaan ke `CaptureSaveWorker`, lalu langsung lanjut ke frame berikutnya. Penulis itulah yang menulis gambar WebP dan JSON ke `artifacts/results/`, lalu menambahkan satu baris ke `outbox.db`. Alasannya terukur: satu janjang memakan ~590 ms untuk disimpan pada frame

2448×2048, dan selama itu penilaian **berhenti**: frame dibuang diam-diam, jejak ByteTrack putus, layar membeku.

4. **Pengiriman lewat antrean.** `OutboxRetryWorker` memeriksa `outbox.db` setiap detik dan mengirimnya ke konsol. Bila konsol sedang tidak aktif, data menunggu di antrean tanpa hilang.
5. **Pencatatan di konsol.** Konsol menyimpan data ke `state/console.db` (SQLite). Layar operator membaca dari indeks ini dan **tidak pernah** memindai folder, karena pemindaian berkala akan menyita I/O yang dibutuhkan proses penilaian.
6. **Unggah berkala ke cloud.** `BatchUploadWorker` mengunggah foto ke Cloudflare R2 dan datanya ke API cloud setiap jam. Jalur ini terpisah dari langkah 4 dan boleh tertunda.

Rancangan ini mengikuti kondisi pabrik: listrik padam, internet terputus, dan container dimulai ulang adalah kejadian rutin. Karena data selalu tersimpan di disk lebih dulu, gangguan hanya menyebabkan keterlambatan, bukan kehilangan data.

5. Alur Kunjungan Truk

Data per janjang adalah rincian. Yang masuk ke pembukuan adalah **kunjungan truk**:

#	Kejadian	Proses di sistem
1	Truk bermuatan naik timbangan	timbangan → konsol → AutoERP, tahap <code>gate</code> : berat bruto dan jam masuk
2	Operator memasang truk ke line	konsol memberi tahu line: janjang berikutnya dicatat atas nama truk ini
3	Buah dituang dan dinilai	data janjang tersimpan di <code>console.db</code> ; AutoERP belum menerima apa pun
4	Operator melepas truk dari line	konsol → AutoERP, tahap <code>grading</code> : total, ACC, REJ, dan persentasenya
5	Truk kosong naik timbangan	konsol → AutoERP, tahap <code>departed</code> : berat tara dan jam keluar
6	AutoERP memproses	neto = bruto – tara, potongan, harga, lalu Purchase Receipt

Satu kunjungan dikirim sebagai satu pesan `upsert_visit` , sebanyak tiga kali sesuai tahapannya.

Setiap kiriman **menggantikan** bagian yang dibawanya. Karena itu bagian yang belum tersedia **tidak dikirim sama sekali**: bagian kosong akan menghapus data yang sudah ada di AutoERP.

Janjang yang ditolak dinaikkan kembali ke truk dan ikut ditimbang saat truk keluar. Beratnya masuk ke tara, sehingga otomatis tidak ikut dibayar.

6. Struktur Repositori

Folder utama

Folder	Isi	Kapan dibuka
<code>src/palmgrade/</code>	seluruh kode Python	setiap perubahan perilaku
<code>tests/</code>	unit, e2e, integration	setiap perubahan perilaku: test ditulis lebih dulu
<code>docs/</code>	dokumen teknis, termasuk dokumen ini	saat membutuhkan rincian
<code>scripts/</code>	kiosk, data contoh, build engine, smoke test, pembuat PDF	sese kali
<code>config/camera/</code>	<code>hikrobot.mfs</code> : setelan kamera, termasuk fps	saat mengubah fps atau exposure
<code>sdk/</code>	SDK kamera Hikrobot (MVS), disertakan agar build Docker tidak butuh internet	hampir tidak pernah
<code>models/release/</code>	berkas model YOLO (<code>.pt</code>)	saat mengganti model
<code>engines/</code>	engine TensorRT hasil build per GPU	tidak diubah manual; diisi <code>make build-engine</code>
<code>images/</code>	<code>sample_sawit.jpg</code> , gambar contoh untuk <code>CAMERA_TYPE=photo</code>	saat menguji tanpa kamera
<code>artifacts/</code>	hasil runtime: foto, JSON, dan <code>outbox.db</code>	saat memeriksa bukti penilaian
<code>state/</code>	<code>console.db</code> , <code>erp_outbox.db</code> , <code>upload_manifest.db</code>	saat memeriksa isi antrian

Isi `models/` , `engines/` , `artifacts/` , dan `state/` **tidak disimpan di git**: ukurannya besar, berbeda di tiap mesin, dan sebagian bersifat rahasia.

Isi folder `src/palmgrade`

Kode disusun berlapis, dan urutan lapisannya **tidak boleh dilompati**: `routes` → `controllers` → `services` → `repositories` / `pipelines` / `integrations` .

Folder	Tanggung jawab	Contoh isi
<code>core/</code>	setelan dan perakitan dependensi	<code>config.py</code> (seluruh env var), <code>dependencies.py</code> , <code>constants.py</code>
<code>routes/</code>	daftar endpoint HTTP saja	<code>console.py</code> , <code>internal.py</code> , <code>health.py</code>

Folder	Tanggung jawab	Contoh isi
<code>controllers/</code>	menerima request dan meneruskannya ke service	<code>capture_controller.py</code>
<code>services/</code>	alur bisnis	<code>console_service.py</code> , <code>erp_queue.py</code>
<code>repositories/</code>	baca-tulis disk dan SQLite	<code>console_repository.py</code> , <code>capture_repository.py</code>
<code>pipelines/</code>	inferensi YOLO	<code>model_registry.py</code> , <code>realtime_inspection_pipeline.py</code>
<code>workers/</code>	proses latar yang berjalan terus	capture, processing, capture_save (penulis bukti), display, outbox, batch upload, worker ERP
<code>integrations/</code>	sistem luar	<code>camera/</code> , <code>erp/</code> , <code>notifications/</code> , <code>storage/</code> , <code>upload/</code> , <code>outbox/</code> , <code>scheduler/</code>
<code>domain/</code>	aturan murni tanpa I/O	<code>working_day.py</code> , <code>ffb_source.py</code> , <code>vision_event.py</code> , <code>plate.py</code>
<code>schemas/</code>	bentuk request dan response (Pydantic)	<code>internal_schema.py</code>
<code>plc/</code>	MC Protocol ke CPU Mitsubishi (Modbus ke coupler masih bisa dipilih); berdiri sendiri, nonaktif secara bawaan	-
<code>license/</code>	penjaga langganan (Ed25519), opsional	<code>manager.py</code> , <code>guard.py</code>
<code>static/</code>	<code>console.html</code> : layar operator dalam satu berkas , tanpa build dan tanpa CDN	-

Dua folder yang sering disalahpahami:

- **`domain/` berisi aturan yang harus benar tanpa bergantung pada apa pun.** Isinya fungsi murni: menerima nilai dan mengembalikan nilai, tanpa akses disk maupun HTTP. Bagian ini paling mudah diuji sekaligus paling mahal bila keliru, contohnya `working_day.py` , yang menentukan satu shift malam masuk ke tanggal kerja yang mana.
- **`console.html` sengaja dibuat satu berkas tanpa framework.** Layar ini harus tetap berfungsi saat internet terputus; halaman yang memuat library dari CDN akan kosong tepat pada saat paling dibutuhkan.

Berkas di akar repositori

Berkas	Fungsi
<code>Makefile</code>	seluruh perintah dijalankan lewat sini: <code>make up</code> , <code>make restart</code> , <code>make logs-1</code> , <code>make console</code>
<code>Dockerfile</code>	resep image (torch CPU untuk pengembangan, CUDA untuk pabrik)
<code>docker-compose.yml</code>	empat service: tiga line dan konsol
<code>docker-compose.override.yml</code>	penyesuaian khusus satu mesin; dibaca compose otomatis dan tidak masuk git
<code>.env.example</code>	contoh seluruh setelan; <code>.env</code> yang sebenarnya tidak masuk git
<code>requirements.txt</code>	dependency Python dengan versi terkunci
<code>CLAUDE.md</code>	ringkasan aturan untuk AI agent
<code>README.md</code>	cara memasang dan menjalankan secara lengkap

7. Menjalankan Sistem

Di Mac: pengembangan konsol

```
cd autograde
make operator      # sekali: akun lokal (tanya email + nama + sandi)
make console       # http://127.0.0.1:8100/console
```

Konsol berjalan native, tanpa Docker, di port 8100, port 8000 dipakai AutoERP lokal. Target `make up` dan `make up-dev` **tidak** dipakai di Mac karena membutuhkan SDK kamera dan GPU NVIDIA.

Menjalankan test:

```
.venv/bin/pytest tests/unit      # jangan tambahkan -q; pyproject.toml sudah memasangnya
.venv/bin/ruff check <daftar berkas di .github/workflows/ci.yml>
```

Di PC pabrik: Linux dengan GPU

```
make up              # build image, bangun engine TensorRT, jalankan tiga line dan konsol
make operator-docker # sekali: akun operator untuk login konsol (di dalam container)
make ps              # status container
make logs-1          # log line 1
make restart         # setelah mengubah kode (kode di-bind-mount, tanpa rebuild)
make start           # setelah mengubah .env atau docker-compose.override.yml
```

Pedoman singkat: **ubah kode** → `make restart` ; **ubah setelan** → `make start` ; **ubah dependency atau Dockerfile** → `make up` .

8. Hal yang Wajib Diwaspadai

Setiap butir berikut pernah menyebabkan kehilangan waktu berjam-jam.

1. **Laju frame kamera hanya diatur di satu tempat:** `config/camera/hikrobot.mfs`. Berkas ini dikirim ke kamera setiap kali terhubung, lalu lajunya dibaca kembali dari kamera. `CAMERA_FPS` di `.env` hanya cadangan untuk sumber yang tidak dapat melaporkan lajunya (webcam, berkas video).
2. **Port 8000 di Mac adalah AutoERP, bukan konsol.** Konsol di Mac selalu di port 8100. Di PC pabrik, konsol di dalam Docker memang di port 8000.
3. **BACKEND_URL wajib menunjuk API lokal.** Pernah diarahkan ke API cloud, dan produksi menerima sekitar 1.098 event uji dalam sehari.
4. **Label produksi adalah ACC dan REJ**, bukan `MATANG` atau `MENTAH`. Nilai lain ditolak dengan status 400 saat diterima, karena angka ini menentukan pembayaran.
5. **net_kg selalu dihitung ulang, tidak pernah diterima apa adanya.** Neto yang berbeda lebih dari 1 kg dari `gross_kg - tare_kg` ditolak. Dua sumber angka yang diam-diam berbeda adalah cara paling mudah untuk salah bayar berbulan-bulan.
6. **Tanggal kerja dihitung saat data diterima, lalu disimpan.** Pabrik beroperasi sekitar 20 jam sehari dan melewati tengah malam; menghitung tanggal dari waktu pembacaan akan memecah satu shift menjadi dua hari.
7. **Nama image GHCR tetap `palgrade-vision`** walaupun repositori sudah bernama `autograde`. Mengganti nama image membuat PC pabrik terus meminta nama lama, dan pembaruan berhenti tanpa pesan error. Ketentuan ini dijaga oleh test.
8. **Berkas `.mfs` di PC pabrik dilacak git.** Jalankan `git checkout config/camera/hikrobot.mfs` sebelum `git pull` bila berkas itu sempat diubah untuk pengujian.

9. Aturan Kerja Tim

- **Setiap perubahan melalui pull request.** Branch baru → PR *squash* ke `staging` → PR *merge commit* ke `main`. Tidak ada commit langsung ke `staging`.
- **Judul dan isi PR ditulis dalam bahasa Inggris.** Pesan commit boleh berbahasa Indonesia.
- **Test ditulis sebelum kode.** Pastikan test gagal dengan alasan yang benar, baru perbaiki kodenya. Test yang tidak pernah gagal tidak membuktikan apa pun.
- **Unit test tidak boleh memuat torch, OpenCV, atau hardware.** CI berjalan tanpa GPU; pengujian yang membutuhkan komponen berat ditempatkan di `tests/e2e/`.
- **Komentar kode ditulis dalam bahasa Inggris, singkat, dan menjelaskan alasan:** bukan mengulang apa yang sudah terbaca dari kodenya.
- **Commit tidak mencantumkan Co-Authored-By** atau referensi AI apa pun.
- **Selama Opsi B berjalan tidak ada deploy dan tidak ada tag rilis** `vX.Y.Z`.

10. Rujukan Dokumen

Kebutuhan	Dokumen
Ringkasan aturan untuk AI agent	<code>CLAUDE.md</code>
Alur rinci, diagram, dan seluruh invariant beserta alasannya	<code>docs/overview.md</code>
Batas antar lapisan kode	<code>docs/architecture.md</code>
Daftar endpoint, event, dan variabel lingkungan	<code>docs/backend-overview.md</code>
Pemasangan dari nol di PC pabrik	<code>docs/SETUP.md</code>
Spesifikasi dan setelan kamera	<code>docs/camera-spec.md</code>
Integrasi PLC dan urutan commissioning	<code>docs/plc-mc-handoff.md</code> , <code>docs/plc-integration.md</code>
Kontrak integrasi dengan AutoERP	<code>../autoerp/docs/autograde-integration.md</code>
Status pekerjaan terkini	<code>../docs/PROGRESS-AUTOGRADE-AUTOERP.md</code>
Membuat ulang PDF dokumen ini	<code>scripts/md_to_pdf.py</code>

11. Glosarium

Istilah	Arti
TBS	Tandan Buah Segar: buah sawit yang baru dipanen
Janjang / tandan	satu buah sawit utuh; satuan yang dinilai kamera
PKS	Pabrik Kelapa Sawit
ACC / REJ	diterima / ditolak; hasil penilaian AI
Tangkai panjang	buah diterima tetapi tangkainya terlalu panjang, menambah berat tanpa menambah minyak
Mentah	buah belum matang dengan kandungan minyak rendah
Brondolan	buah lepasan yang rontok dari tandan
Line	satu conveyor dengan satu kamera; terdapat tiga line
Bruto / tara / neto	berat truk bermuatan / truk kosong / selisihnya, yaitu berat yang diterima pabrik
Sortasi	proses penilaian mutu buah
Potongan	pengurangan nilai pembayaran karena mutu buah
Sumber TBS	Internal (kebun sendiri) atau External (pembelian dari pemasok)
Kunjungan	satu siklus truk: masuk, bongkar, keluar
Outbox	antrean di disk untuk data yang belum berhasil terkirim

12. Langkah Pertama

1. Baca bagian **Critical Rules** di `CLAUDE.md` .
2. Buat operator dengan `make operator` , jalankan `make console` , lalu masuk dengan email dan sandi itu dan coba layar operator.
3. Buka `src/palmgrade/workers/frame_processing_worker.py` : di sinilah janjang diubah menjadi angka.
4. Buka `src/palmgrade/domain/working_day.py` : contoh aturan murni yang ringkas di `domain/` .
5. Jalankan `.venv/bin/pytest tests/unit` dan pastikan seluruhnya lolos sebelum mulai mengubah kode.

Bila menemukan hal yang membingungkan atau tampak keliru, **catat sebagai temuan**, jangan dianggap "memang begitu".

Riwayat Revisi

Versi	Tanggal	Perubahan
1.2	15 September 2026	Login email + sandi: akun dari AutoERP (<code>AutoGrade Operator</code>) atau lokal, diverifikasi offline
1.1	15 September 2026	Login konsol (Fase 4): gerbang PIN, akun lokal lewat <code>make operator</code>
1.0	14 September 2026	Rilis pertama

Akhir dokumen